MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : 01

Lab Report on : Comparing Abstract Classes and Interfaces for Multiple Inheritance in Java

Course Title : Software Engineering and Project Management Lab

Course Code : ICT 3108

Submitted By	Submitted To
Joy Sarker ID: IT23022 3rd Year, 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU.	Dr. Ziaur Rahman Professor, Dept. of ICT, MBSTU.

Date of Performance:

Date of Submission:

Lab no 01:

Lab Name: Compare abstract class and interface in terms of multiple inheritence, When would you prefer to use an abstract class and when an interface

In OOP, multiple inheritence means that a class can inherit from more than one parent class or multiple types.

Comperison between abstract class and interfaces in terms of multiple inheritence

Feature	Abstract class	Interface
Inheritence type	Single inheritence only	multiple inheritence is supported
Keyword	abstract class	Interface
method	Can have both abstract and concrete method	Can have abstract methods, default, static and private methods.
Construc-tor	Can have constructor	Can not have constructor
Access Modifier	Methods can be public, protected private or, default	All method are implicitly public abstract.
Fields	Can have instance variables and constants	Only public static final constants

When to use abstract class:

(i) You want to share code (common implementation methods)

(ii) You want to control access modifiens

(iii) You need constructor.

(iv) When the classes are closely related.

When to use interface:

(i) You need multiple inheritence

(ii) You are defining capabilities

(iii) You are using functional programming patterns

(iv) When the classes can be related

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : 02

Lab Report on : Implementing Encapsulation for Secure Data Handling

Course Titie : Software Engineering and Project Management Lab

Course Code : ICT 3108

Submitted By	Submitted To
Joy Sarker ID: IT23022 3rd Year, 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU.	Dr. Ziaur Rahman Professor, Dept. of ICT, MBSTU.

Date of Performance:

Date of Submission:

Lab no: 2

Lab Name: How does encapsulation ensure data security and integrity? show that with a BankAccount class using private variables and validate method such as set account number (string), setinitial Balance(double) that reject null, negative and empty variables

⇒ Encapsulation a core principal of object oriented programming. Enhances data security and integrity by bundling data and methods that operate on that data within a single unit and restrict direct access to the data. It protects data from.

It protect data from -

(i) Direct unauthorised access
(ii) Invalid or harmful input
(iii) Inconsistent object states

Work process of encapsulation is given below with a Bank Account Class

code:

```
public class BankAccount{
private double balance;
private set AccountNoumber.(string Account Number)
{ if(accountNumber=null || accountNumber.trim.
                                    isEmpty ())

{ System. Out. println ("Invalid number: ");}

else {
      this. accountNumber = = account Number;
      }

 }
public void setInitial Balance(double balance){
 if(balance<0)
  System. out. println("Invalid balance");
  else
      this. balance =balance.
 }
public string getAccountNumber(){
    return accountNumber;
 }
```

```
public double getbalance() {
        return balance;
        }

public void deposit (double amount) {
    if (amount<=0) {
    System.out.println ("Invalid amount");
    }
else{ balance += balance;
    system.out.println ("Deposited : "tamount);
    }
}
}
public class BankApp {
public static void main(string[] args) {
BankAccount acc = new BankAccount ();
acc.set Account Number("");
acc.set Account Number ("Acc 23042");
```

```
acc.setInitialBalance(-1000)
acc.setInitialBalance(1000);
acc.deposit(-500)
acc.deposit(500)
System.out.println("Final Balance: "acc.
                    getBalance());
    }
}
```

So the benefit of using encapsulation here is:

(i) private variables → prevent direct external manipulation.

(ii) validate setters → Reject and input such as: null, empty, negative etc.

(iii) Controlled methods → Ensures proper business logic(deposit)

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : 04

Lab Report on : Using JDBC to Execute SELECT Queries and Handle Exceptions in Java

Course Title : Software Engineering and Project Management Lab

Course Code : ICT 3108

Submitted By	Submitted To
Joy Sarker ID: IT23022 3rd Year, 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU.	Dr. Ziaur Rahman Professor, Dept. of ICT, MBSTU.

Date of Performance:

Date of Submission:

Lab no : 04

Lab Name: Describe how JDBC manages communication between a java application and a relation database outline the steps involved in executing a SELECT query and fetching results. Include error handling with try-catch and finally blocks.

JDBC is an API thatallows java application with to interect with relational database like MySQL postgresql and oracle etc.

How data base JDBC manages communiction:

JDBC acts as a bridge between java and the data using:

(i) DriverManagen → loads the JDBC driver
(ii) Connection → estabishes the session
(iii) statement → sends SQL commands
(iv) ResultSET → holds the data returned from queries

Here's a concise outline of the steps to execute a SELECT query using JDBC with error handling.

1. Load the JDBC Driver
2. Establish a connection
3. prepare SQL query
4. set parameters.
5. Execute the query
6. Process the Resultset
7. Handle Exceptions
8. closure resource in finally block

Example of catch and finally block:

```
try {
    // set parameters, execute,
    }
catch (SQLException e) {
    // handle sql error
    }
finally {
    // close. resultset, statment
    }
```

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : 05

Lab Report on : Servlet Controller in Java EE: Managing Model-View Flow with JSP

Course Title : Software Engineering and Project Management Lab

Course Code : ICT 3108

Submitted By	Submitted To
Joy Sarker ID: IT23022 3rd Year, 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU.	Dr. Ziaur Rahman Professor, Dept. of ICT, MBSTU.

Date of Performance:

Date of Submission:

Lab no. 05:

Lab Name: In a java EEE application, how does a servlet controller manage the flow between the model and the view? Provide a brief example that demonstrates. Forwarding dataframe servlet to a jsp and rendering a response.

⇒ In a java EEE application, a servlet controlling manages the flow between the model and the view by:

① Receiving requests from the client

② calling model classes to process data on fetch from the database.

③ setting attributes on the request scope.

Flow overview:

Client → servlet → model → jsp (view) → client

Example: Student Info

public class Student {

```
public string name;
private int age;
public student(string name, int age){
  this.name = name;
  this.age  = age;
public string getName(); {
    return name;}
public int getAge(){
              return age; }
```

2. jsp (view):

```
<% @ page import. your. package. student %> ;
<%
  students student = (student) request. getAttribute "
("student Data "); %>

< html>
< head > <tittle> student info </ tittle> </
 head >
< body >
<h2> student details </h2>
<p> Name: <%= student.getName () %>
    </p>
```

```
<p> Age: <%= student.getAge() %> </p>
</body>
</html>
```

3. Controller (Servlet):

```
@WebServlet("/student")
public class StudentServlet extends Servlet{
protected void doget (pservlet Request request)

throws, servlet Exception IOException{
student students new student("Joy", 22);
request.setAttribute("studentData", student);

RequestDispatcher dispatcher = reques.getReq
                               est Dispatcher
                               ("student.jsp");

dispatcher forward (request.response);
  }
}
```

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : ০৬

Lab Report on : ResultSet in JDBC: Fetching Data with next(), getString(), and getInt()

Course Title : Software Engineering and Project Management Lab

Course Code : ICT 3108

Submitted By	Submitted To
Joy Sarker ID: IT23022 3rd Year, 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU.	Dr. Ziaur Rahman Professor, Dept. of ICT, MBSTU.

Date of Performance:

Date of Submission:

Lab no : 06

Lab name: What is a Resultset in JDBC can how it is used to retrieve data from a MYSQL database? Briefly explain the use of next(), getString() and getInt() methods with an example.

In JDBC, a Resultset is an object that holds the data returned from executing a select query on a returned from executing a select query on a relational data such as: MYSQL. It represents a table of data. Each row in the result set can be accessed one at a time, using a cursor that move forward.

How it retrieves data in given below:

(rs.next() moves the first row)

- Establish a database connection
- Create a statement
- Execute the query
- Iterate through the Resultset
- Retrieves column values

- process the retrieve data.
- class Resources.

- Use the common methods with example:

Next(): Moves the cursor to the next row. Returns false when there are no more rows.

getString(): Retrives the values of the specified column as a string.

getInt(): Retrieves the value of the specified column as an int:

Example:

```
String query = "select id, name, mark. from
                students";

Result set rs = stml. execute Query(query)
while(rs. next()){
int id = rs. getstring ("name");
 string name = rs. getstring ("Name");
 it. marks = rs. getInt ("marks");

system. out. println(Id + "I "+nam + "I"
                    "+" mark);
  }
```

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : ~~০৪~~ ০৯

Lab Report on : Secure and Efficient Database Insertion in JDBC Using PreparedStatement

Course Title : Software Engineering and Project Management Lab

Course Code : ICT 3108

Submitted By	Submitted To
Joy Sarker ID: IT23022 3rd Year, 1st Semester Session: 2022-2023 Dept. of ICT, MBSTU.	Dr. Ziaur Rahman Professor, Dept. of ICT, MBSTU.

Date of Performance:

Date of Submission:

Lab no : 07

Lab name : How does JPA manage the between Java objects and relational tables ? Explain with an example an example using @Entity @Id and @Generated value annotations. Discuss the advantage of using JPA over raw JDBC.

⇒ Entity definition

. Java classes are marked as JPA entities using the @Entity annotations. This indicates that the class represents a table in the database

Field to column mapping :

. By default, JPA maps fields in an entity class to column in the corresponding data base table with the same time.

Relationship mapping :

JPA provides annotation to manage relationships between entities, which correspond to relationships between tables in the data base.

Example with @ Entity, @ ID and @ Generated value annotation:

```
import jakarta.persistence.Entity;
import jakarta.persistence.ID;
import jakarta.persistence.generatedvalue;

@Entity
   public class student{
          @ID
          private int id;
          private string name;
   public  int getID(){
                 return id;}

public void setID(){(int id)}
          "this.Id = id";}

 public string getName(){
                 return name;}

public void setName(string name){
   this.name = name;}
}
```

The advantages of using JPA over pow JDBC for database insenaction java application.

(i) Object-Relational mapping

(ii) Reduced Boilenplate code.

(iii) Database Independence

(iv) Reduced Boilenplate code

(v) Improve maintainability

(vi) Enhanced productivity